

Resumen de arquitectura

Arquitectura, flujo de datos y modelo de seguridad — para arquitectos empresariales y técnicos

Producto	Anthropic AI Assistant para Qlik Sense (extensión + proxy)
Versión	v0.5.0 · compilación 34
Autor	Miguel Angel Baeyens
Fecha	25 de julio de 2026
Idioma	Español

Aviso — demostración experimental

Esta extensión es un desarrollo de demostración y no un producto oficialmente soportado por Qlik. Ni Qlik ni el autor asumen responsabilidad alguna por cualquier incidencia.

Al hacer una pregunta, los datos del gráfico seleccionado salen del entorno hacia un LLM (Anthropic, a través del proxy) — salvo que use un modelo local, en cuyo caso los datos permanecen en su infraestructura. No lo utilice con datos sensibles, regulados o personales sin autorización expresa.

1. Resumen ejecutivo

El Anthropic AI Assistant es una extensión de visualización de Qlik Sense que permite analizar los datos del gráfico seleccionado mediante la familia de modelos Claude de Anthropic (o un modelo local vía Ollama). Desde la v0.5.0 el sistema es una pareja endurecida: la extensión (cliente) y un proxy Node.js (servidor).

- Plataforma destino: Qlik Sense Enterprise on Windows (QSEoW, qlik-sense ≥ 2025). Qlik Cloud queda fuera de alcance.
- Patrón de integración: el navegador NUNCA llama directamente a `api.anthropic.com`. Todo el tráfico pasa por el proxy, que guarda la clave en el servidor y valida la sesión de Qlik del llamante.
- Sin paso de compilación en el cliente: JavaScript (AMD) cargado mediante el sistema RequireJS de Qlik.
- El proxy es un runtime Node.js independiente, versionado por separado, desplegable como servicio de Windows.

2. Arquitectura de módulos (extensión)

Todos los módulos siguen el patrón RequireJS `define([deps], factory)`. El punto de entrada `AnthropicExtension.js` delega en `js/main.js`, que expone `paint(element, layout)`, invocado por Qlik en cada ciclo de renderizado. La inicialización de UI/selección se realiza una sola vez por instancia (guarda de primera ejecución sobre el widget flotante); las propiedades (modelo, URL de proxy, URL local, nivel de log) se aplican en cada llamada a `paint()`.

Librerías empaquetadas (sin CDN): `marked` (Markdown → HTML) y `DOMPurify` (saneado del HTML). `CryptoJS` y `js/security.js` se han ELIMINADO: al no haber clave en el navegador, ya no hay nada que cifrar.

Módulo	Responsabilidad
<code>AnthropicExtension.js</code>	Punto de entrada (shim RequireJS hacia <code>js/main</code>).
<code>js/main.js</code>	Ciclo <code>paint()</code> de Qlik, panel de propiedades, guarda de primera ejecución, validación de config.
<code>js/ui-controller.js</code>	Panel flotante, hilo de conversación, selección de gráficos, ensamblado de la petición, control de una única petición en curso (<code>busy/abort</code>) y guarda de payload.
<code>js/data-collector.js</code>	Extrae el hipercubo del objeto seleccionado y recoge el contexto de la app (tablas/campos + ítems maestros) una vez por sesión (memoización <code>race-safe</code>).

Módulo	Responsabilidad
js/data-format.js	Compacta y formatea los datos para el LLM; estima tokens y recorta filas para no superar el presupuesto.
js/anthropic-api.js	Construye el mensaje, resuelve el ÚNICO transporte (proxy) y realiza el POST con credentials:include para reenviar la sesión de Qlik. Soporta streaming (SSE).
js/chart-builder.js	Interpreta la especificación de gráfico de Claude, previsualiza y añade el gráfico a la hoja.
js/formatting.js	Renderiza la respuesta Markdown a HTML saneado (DOMPurify, fail-closed).
js/config-validate.js	Valida la configuración al iniciar; los problemas se muestran en el panel en lugar de romper el render.
js/log.js	Envoltura de consola con niveles (ERROR/WARN/INFO/DEBUG) controlados por config.LOG_LEVEL.
js/template.js	Marcado HTML del panel (inyectado por ui-controller).
js/config.js	Única fuente de verdad: modelo, endpoints de proxy, tokens, límites, niveles de log y prompt.

3. Flujo de petición de extremo a extremo

- Selección: el click se detecta vía la clase DOM `qv-object-<id>`; se extrae el hipercubo del objeto (hasta el tope de celdas permitido).
- Contexto de la app (solo en la primera petición): `getAppContextCached()` obtiene nombres reales de tablas y campos e ítems maestros (dimensiones/medidas con sus expresiones) y los cachea durante la sesión.
- Formateo y presupuesto: se compactan los datos y se estiman tokens (~4 caracteres por token), recortando filas según `MAX_ROWS`.
- Ensamblado: `ui-controller` construye `{ userPrompt, chartData, context, systemPrompt, history }`. Si el payload supera ~65 KB se pide confirmación; por encima de 1 MB se bloquea.
- Guarda de contexto: si la estimación supera la ventana del modelo, el usuario puede truncar los datos o cancelar.
- Transporte: una única ruta al proxy (`config.API.PROXY_URL` para modelos hospedados; `config.API.LOCAL_URL` para modelos locales). El navegador no lleva clave alguna.
- Llamada: POST al proxy con `credentials:include`, de modo que el navegador adjunta automáticamente la cookie de sesión de Qlik. El proxy valida la sesión, inyecta la clave y reenvía al upstream.
- Render: la respuesta se renderiza como Markdown saneado en el hilo, con botón de copia. Con streaming, token a token.
- Gráfico sugerido (opcional): `chart-builder` previsualiza y, en modo edición, añade el objeto a la hoja como el usuario autenticado.

4. El proxy endurecido

El proxy (carpeta `proxy/`, Node.js + Express) es el punto donde vive la clave y donde se autentica al llamante. Sus capas de endurecimiento (especificaciones P01–P07):

Capa	Función
Custodia de credenciales (P01)	La clave de Anthropic se guarda en el servidor (variable de entorno <code>ANTHROPIC_API_KEY</code>), se inyecta por petición y se elimina cualquier cabecera de clave enviada por el cliente.

Capa	Función
Autenticación del llamante (P02)	Valida la sesión de Qlik contra la API de QPS por TLS mutuo antes de cualquier llamada upstream. En el despliegue same-site el navegador envía la cookie X-Qlik-Session automáticamente (un virtual proxy con prefijo usa X-Qlik-Session-<prefijo>).
Concurrencia y resiliencia (P03)	Control de admisión con límites global y por usuario, cola FIFO acotada (503 + Retry-After al desbordar), backpressure en streaming y drenado ordenado al apagar.
Validación y allowlist (P04)	Validación de esquema por ruta, tope de tamaño de cuerpo y lista blanca de modelos en el servidor: se rechaza lo inválido/excesivo/no permitido.
Transporte y CORS (P05)	Allowlist estricta de orígenes, cabeceras de seguridad, suelo de TLS moderno y límite de tasa por IP.
Observabilidad y servicio (P06)	Logs JSON estructurados con niveles, log de auditoría separado (sin cuerpos ni secretos), /health · /ready · /metrics, validación de configuración al arrancar y wrapper de servicio de Windows.

5. Especificación de la API (proxy)

Rutas

POST /api/anthropic → reenvía a api.anthropic.com/v1/messages (la clave la inyecta el proxy)
POST /api/ollama → reenvía a un servidor Ollama local (formato OpenAI chat-completions)
GET /health → liveness GET /ready → readiness GET /metrics → contadores

Petición del cliente al proxy

POST <config.API.PROXY_URL>
Content-Type: application/json
Cookie: X-Qlik-Session=<sesión> (enviada automáticamente por el navegador, same-site)
(el navegador NO envía ninguna clave de API; el proxy la añade en el servidor)

Prompt de sistema por defecto: “You are a business analyst and expert Qlik Sense user. Be concise. Always aggregate the data and show absolute values and percentages. Focus on insights that would help business decision making. Present your analysis in a structured format with bullet points for key findings.”

Parámetro	Valor
Endpoint del proxy (hosted)	https://<host>:3000/api/anthropic (por defecto en config.js: localhost)
Endpoint del proxy (local)	https://<host>:3000/api/ollama
Modelo predeterminado	claude-haiku-4-5
Modelos disponibles	claude-haiku-4-5, claude-sonnet-4-6, claude-opus-4-8, Ministral 3 8B/3B (local)
max_tokens	4000
Tiempo de espera	60 000 ms (hosted) · 300 000 ms (modelo local)
Ventana de contexto (aprox.)	200 000 tokens (hosted) · 8 192 (Ministral local)

6. Modelo de seguridad

¿Dónde vive la clave de API?

- En el proxy, en el servidor (variable de entorno ANTHROPIC_API_KEY). El navegador nunca la ve ni la almacena.
- El proxy elimina cualquier cabecera de clave que un cliente pudiera enviar e inyecta la suya propia por petición.

- Ya no hay CryptoJS, ni localStorage, ni passphrase empaquetada: esa superficie de riesgo se ha eliminado por completo.

Autenticación

- El proxy valida de forma independiente la sesión de Qlik del llamante (nunca confía en una identidad afirmada por el cliente) y resuelve el usuario real para límites por usuario y auditoría.
- Carrier por defecto: la cookie de sesión same-site (X-Qlik-Session). Como alternativa explícita se admite la cabecera x-qlik-session para despliegues que emitan un ticket de sesión.

Datos que salen del navegador

- En cada petición: el contenido del hipercubo del gráfico seleccionado, la pregunta del usuario y el historial (hasta 12 mensajes).
- Solo en la primera petición: nombres reales de tablas y campos e ítems maestros con sus expresiones.
- No se envían: credenciales de Qlik hacia el LLM ni datos de otras hojas/apps. Todo el tráfico es HTTPS.
- Con un modelo local (Ollama), los datos van del navegador al proxy y de ahí a Ollama en su propia red: no salen a Internet.

7. Modelo de despliegue

- Extensión (sin build): JavaScript + RequireJS; no requiere npm, transpilación ni bundling en runtime. Los cambios surten efecto al refrescar el navegador. Importar el .zip desde la QMC → Extensiones.
- Proxy: runtime Node.js en el nodo de Qlik. Instalación automatizada con proxy/scripts/setup.ps1 (dependencias de runtime con npm ci --omit=dev, certificado TLS, .env y, opcionalmente, servicio de Windows).
- Despliegue recomendado del proxy: same-site respecto al hub, de modo que el navegador envíe la cookie de sesión de Qlik automáticamente y el proxy la valide contra QPS por TLS mutuo.
- Certificado del proxy: en producción, CA-firmado para el FQDN que use la extensión, de modo que la página de Qlik confíe sin importaciones manuales.

8. Límites y restricciones configurables

Parámetro	Valor	Propósito
MAX_ROWS	1000	Máximo de filas enviadas al LLM por gráfico.
MAX_FETCH_CELLS	50000	Tope de celdas leídas de un hipercubo; más allá se trunca con aviso.
MAX_CELLS_PER_PAGE	10000	Celdas por página de getHyperCubeData.
LARGE_TABLE_CELLS	25000	Umbral a partir del cual se avisa de tabla grande/truncada.
FETCH_PAGE_CONCURRENCY	4	Máximo de peticiones de página concurrentes al motor.
MAX_FIELDS	500	Tope de campos de la lista enviada como contexto.
WARN / MAX_PAYLOAD_BYTES	~65 KB / 1 MB	Aviso de confirmación / bloqueo duro por tamaño de payload.
HISTORY_MAX	12	Mensajes previos enviados por petición.

Las visualizaciones de tipo mapa no están soportadas.

9. Consideraciones de gobierno de datos y red

- Cada análisis con un modelo hospedado implica salida de datos del gráfico hacia un LLM externo (Anthropic); la residencia y retención de esos datos quedan fuera del control directo del cliente.

- El proxy centraliza el control: es el único punto que sale a Internet, aplica allowlist de modelos, límites de tasa, auditoría y validación de la sesión de Qlik.
- Para datos que no deban salir del perímetro, use un modelo local (Ollama) a través del proxy.